SCALABLE AUTOMATION FRAMEWORKS

for distributed systems

STAR EAST

April 26 - May 1, 2026 | #STAREAST
ORLANDO, FL + Online

PRESENTER: Walter Zimerman
Sr. Software Development Engineer | Amazon

About Me

Walter Zimerman

Sr. SDE| *Amazon*

Summary | I am becoming an *AI herder!*

linkedin.com/in/walter-ariel-zimerman/

STAR EAST

My mindset

Automated testing must cover more than just functional tests of components, it needs to do it **easily**, **fast**, and **well**.

To build a system like that is not straightforward...

What we'll see

Definitions:

- Distributed Systems
- Testing, Test Automation, and Testing Framework

Scalable frameworks components

Q&A

Distributed Systems

A **distributed system** is a collection of autonomous computing components that cooperate over a network to present a single, coherent service to users

Distributed Systems

Each component:

- Owns its state
- Executes concurrently
- Can fail on its own schedule.

Network brings its physics:

- Latency
- Jitter
- Packet loss
- Partitions
- Congestion control

Distributed Systems

Correctness, doesn't depend only on internal logic.

Testing must treat communication as a **first class citizen** and validate that the whole exceeds the sum of its independently deployable parts.

Almost every system today...

Single page application calling rest endpoints...	distributed.
Monolithic codebase using cloud storage...	distributed.
Voice assistant chaining speech recognition, Natural language understanding, and a bunch of skill adapters....	distributed.

In 2026, almost all systems, are distributed by accident, or by intent.

Why does this matter for testing?

Boundaries multiply faster than code...

And automated test scalability, is not about throughput... is about cognitive offloading.

The Good news

If we do things correctly, distributed architecture gives us leverage points.

A test framework that understand message, latency, and component orchestration, turns them into an advantage.

Mental model

Embrace distributed reality, treat every boundary as an opportunity, and design your framework so that adding another component costs you days, not weeks, including test coverage.

Testing

ISTQB: Testing is the process of evaluating a system or component to determine whether it meets specific requirements, and to identify defects.

Testing

Two implications:

- Testing is an activity, not a tool.
- Requirements drive the evaluation.

Does not mention:

- Who does it.
- What language to use.
- If we click or send http requests.

Information gathering, in service of decisions.

Test Automation

The use of software to perform or support testing activities.

It is not magic, nor a silver bullet...

is a power tool for testers.

A test harness

Often we see a harness that invokes code, collects the results, and decorates pipelines with badges.

This is vital, but is only execution and reporting. A “Test execution harness”

Excellent execution harnesses share some traits:

- Determinism
- Observability
- Parallelism
- Useful reporting

All these are important… but are missing many things.

Test frameworks are bigger

An ecosystem that orchestrates people, process, code, data, and infrastructure, so that quality flows continuously.

Test frameworks are bigger

Three overarching thoughts about frameworks:

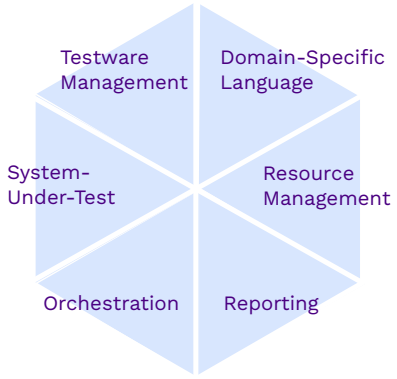
Frameworks bake in scalability as a first principle.

Frameworks encode opinions.

Frameworks are products on their own right.

Components of a test framework

Six cooperating components.



No single side delivers value in isolation.

Testware management

Library that stores **every** artifact your framework consumes.

Dsl Scenarios. Data fixtures. Mock definitions.

Golden screenshots. Reporting templates.

Like everything.... treat it like production code.

Testware management

Considerations

Each item:

- Lives in version control.
- Is reviewed.
- Carries semantic tags.

```
{
 "Tags":{
  "Component":"component name",
  "Type":"Type of test",
  "Priority":"priority"
 }
}
```

Testware management

Considerations:

- Governing schema with mandatory properties.
- Rot prevention policies.
- Branching strategies.

The goal:
Transform scattered scripts into a coherent evolvable corpus.

Domain specific language

A **domain-specific language (DSL)** is the human-readable contract between your intent and the framework's machinery. Its purpose is to let testers describe *what* to verify in, stable vocabulary, while the framework decides *how* to enact those steps.

Domain specific language

```
driver.get("https://example.com/cart");

driver.findElement(By.cssSelector("[data-sku='ABC-123']")).click();
driver.findElement(By.cssSelector("#qty-ABC-123")).clear();
driver.findElement(By.cssSelector("#qty-ABC-123")).sendKeys("2");

driver.findElement(By.id("checkout")).click();


assertEquals("CONFIRMED", driver.findElement(By.id("orderStatus")).getText());
```


Domain specific language

```
{
  "scenario": "new_customer_order_happy_path",
  "given": {
    "actor": "customer:new",
    "cart": {
      "items": [
        { "sku": "ABC-123", "quantity": 2, "stock": "available" }
      ]
    }
  },
  "when": "placeOrder",
  "then": {
    "orderStatus": "CONFIRMED",
    "inventory": "ALLOCATED",
    "confirmationLatency": "≤2s"
  }
}
```

STAR EAST

Domain specific language

```
driver.get("https://example.com/cart");

driver.findElement(By.cssSelector("[data-sku='ABC-123']")).click();
driver.findElement(By.cssSelector("#qty-ABC-123")).clear();
driver.findElement(By.cssSelector("#qty-ABC-123")).sendKeys("2");

driver.findElement(By.id("checkout")).click();


assertEquals("CONFIRMED",
driver.findElement(By.id("orderStatus")).getText());
```

```
{
 "scenario": "new_customer_order_happy_path",
 "given": {
  "actor": "customer:new",
  "cart": {
   "items": [
    { "sku": "ABC-123", "quantity": 2, "stock": "available" }
   ]
  }
 },
 "when": "placeOrder",
 "then": {
  "orderStatus": "CONFIRMED",
  "inventory": "ALLOCATED",
  "confirmationLatency": "≤2s"
 }
}
```


Domain specific language

Design challenge:

- Rich enough to express complex scenarios.
- Constrained enough to keep parser simple, and execution deterministic.

Heuristic: Mirror business vocabulary rather than technical details.

Domain specific language

Guardrails and pitfalls:

- Avoid arbitrary scripting in your dsl.
- Provide expansion points so that power users can extend behaviour.
- Avoid coupling with a specific technology.

A well designed DSL freezes intent, and lets implementation pivot.

Resource Management

The set of services that provisions, isolates, routes, and disposes the infrastructure each tests needs.

Resource Management

Consideration 1: Per-run vs per-test allocation.

Some suites fire test cases against a shared stack.

Others, depend clean environments for each test.

Resource Management

Consideration 2: Creation and routing of components.

Consider some form of intelligent router that can inject environment variables, secrets, etc.

All allocation metadata is exported to the observability stack.

STAR EAST

Resource Management

Consideration 3: Preconditions and data seeding.

Resource management owns the factories that inject customers, products, or feature flags.

When a test finishes, the same services purges data to keep things clean.

Resource Management

Consideration 4: Cost and sustainability.

Elastic infrastructure is a double edged sword.

Create budget alarms that fire before invoices do.

STAR EAST

Resource Management

Consideration 5: Observability hooks.

Resource management is dynamic.

Emit events for traceability and actions.

Resource Management

Resource management becomes a mini “Platform as a service” dedicated to testing.

Reporting

Reporting translates low level details into layered visualizations tailored to diverse audiences.

Layer 3 - Business KPIs
Layer 2 - Aggregated Dashboards
Layer 1 - Task logs, and artifacts

Task logs, and artifacts

This layer contains **granular evidence.**

Request / Response pairs Screenshots Trace spans Service Stats

These items belongs to engineers debugging a specific failure.

Consideration 1: Searchable index.

Consideration 2: Retention policies balance forensic with cost, and compliance.

Aggregated dashboards

This layer aggregates testing data

Pass / fail counts Latency Error Taxonomy

This layer provides an overview about a specific test run.

Consideration 1: Try to integrate in your continuous integration flows.

Consideration 2: Automate annotations of builds, feature flags, code changes.

Business KPIs

Translates technical metrics into customer risks.

SLAs N-th-perncentile metrics

Feed these into release criteria.

Consideration 1: Should help stakeholders make a go / no go decision.

Consideration 2: Avoid low level details.

Presentation layer

The presentation layer is the UX of the quality data.

Every widget should answer three questions:

- What failed?
- Why did it fail?
- What do I do next?

Final consideration: Resist excessive ornamentation.

Orchestration

The orchestrator is the central platform that **manages** and **coordinates** interactions between the different components of our service.

- Pluggable
- Extensible
- Resilient to failure

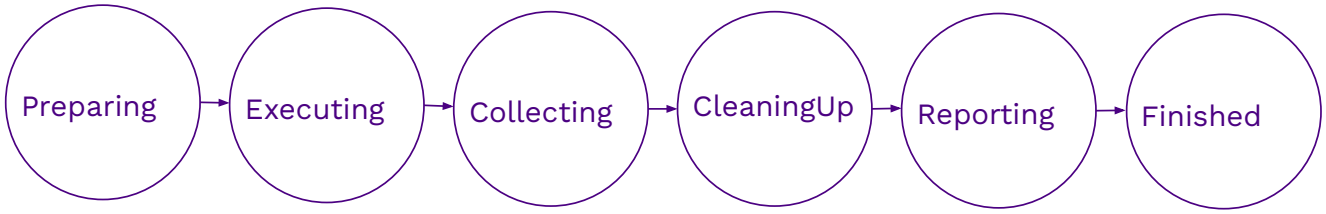
Combining components

The orchestrator first task is **dependency resolution.**

- Queries the testware management platform.
- Spins the environments.
- Register traces ids for observability.

Sequencing matters

Think about a finite state machine:

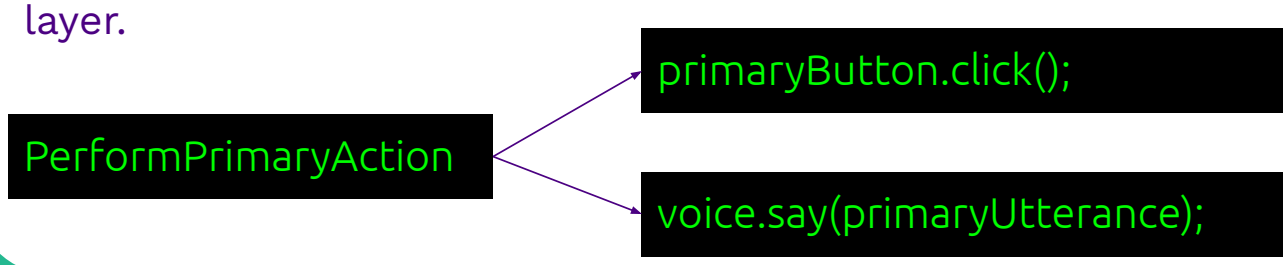


Consideration: Each state emits events, so that other external tools can hook in.

Drivers and abstraction layer

Drivers are the technology specifics.

As the DSL is agnostic, the orchestrator injects a runtime binding layer.



Failure handling and retries

Distributed tests fail for many reasons:

- Genuine bugs.
- Flaky networks.
- Mis-provisioned environments.
- Flaky dependencies.

Tag failures with classifications that make sense to you like `Assertion`, `Infrastructure`, `Unknown`.

Retry metadata flows into reporting.

Resilience and self healing

Considerations:

- Ideally stateless services.
- Keep services behind load balancers.
- Push checkpoints to durable storage.
- Horizontal auto scaling capacity.

Extensibility and governance

The orchestrator MUST be easy to extend.

How do we achieve this?

Design, and expose plugin interfaces for all the components!

Security considerations

Your orchestrator will hold secrets...

Be aware of this, and protect them.

- Integrate with your corporate vault.
- Never pass secrets through the command line ;)
- Do generate audit logs on secret request to trace usage.
- Run your service trough security compliance processes.

Observability of the orchestrator

Dog food your own philosophy. Instrument:

- State machine transitions.
- Structured logs.

System under tests

System under test must expose testability hooks for automated tests to scale.

Black-box testing forces reverse-engineering.

Better is “White-box collaboration”.

Observability points

Three things to think about:

- Distributed tracing.
- Health proves.
- Toggleable faults.

Some extra considerations

- Non Functional Metrics.
- Security and compliance.
- Collaboration contract.
- Toolchain integration.

Observability is the bridge between what we asked, and what actually happened.

Closing remarks

What we saw:

- Distributed systems.
- Precise definitions of testing, and test automation.
- Six sided framework.

Scalable testing, requires a full product, not a script.

Walter Ariel Zimerman

Sr. Software Developer at Amazon

THANK YOU!

Q&A

April 26 - May 1, 2026 | #STAREAST
ORLANDO, FL

Walter Zimerman
linkedin.com/in/walter-ariel-zimerman/